INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, BHOPAL

PROJECT REPORT

EMOTION CLASSIFICATION USING DEEP LEARNING

A text-based emotion prediction system powered by a Bidirectional GRU (BiGRU) neural network

Submitted by	Submitted to
Pranjal Khandelwal Enrollment No.: 25U022031 Harsh Anand Enrollment No.: 25U022032	Prof. Amit Kumar Nandanwar
Department Computer Science and Engineering (CSE)	**Institute** IIIT Bhopal

Academic Project Report
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **“Emotion Classification Using Deep Learning”** is submitted by **Pranjal Khandelwal (25U022031)** and **Harsh Anand (25U022032)** of the Department of Computer Science and Engineering, IIIT Bhopal, to **Prof. Amit Kumar Nandanwar**.

The report describes a text-based emotion classification application built using a Bidirectional Gated Recurrent Unit (BiGRU) model, with a FastAPI backend and a browser-based interface. The project details in this report are based on the supplied project README and its documented implementation.

_____ _____

Project Guide / Faculty Date

DECLARATION

We declare that this report presents the Emotion Classification project and its implementation details to the best of the information available in the project documentation supplied for preparation of this report. Reported model results are reproduced as documented; no additional experimental measurements have been introduced.

Pranjal Khandelwal — 25U022031

Harsh Anand — 25U022032

ACKNOWLEDGEMENT

We express our sincere gratitude to Prof. Amit Kumar Nandanwar for providing academic guidance and the opportunity to work on a practical deep-learning application. We also acknowledge the developers and maintainers of the Hugging Face DAIR.AI Emotion dataset and the open-source Python libraries used in the project, including TensorFlow/Keras and FastAPI.

The project helped connect natural language processing concepts with model development, API engineering, frontend implementation, and cloud deployment. The report is prepared from the project README supplied with the request.

ABSTRACT

Emotion Classification is a natural language processing task that attempts to infer an emotion expressed in a short text. This project implements a web application that accepts an English sentence and predicts one of six emotion categories: sadness, joy, love, anger, fear, or surprise. The model uses a Bidirectional Gated Recurrent Unit (BiGRU) neural network trained on the DAIR.AI Emotion dataset available through Hugging Face.

The pipeline includes label mapping, exploratory data analysis, text normalization, tokenization, sequence padding, class-weighted training, comparison of four recurrent architectures, and evaluation of the selected BiGRU model. The documented final architecture uses a 10,000-word vocabulary, 300-dimensional embeddings, two bidirectional GRU layers, dropout, and a six-unit softmax output. The README reports 0.92 accuracy and 0.21 loss for the selected model; class-wise precision, recall, and F1 scores are not supplied and are therefore identified as further evaluation work.

For practical use, the trained model is exposed through a FastAPI backend and connected to a custom HTML, CSS, and JavaScript interface. The application returns a predicted label, confidence score, and probability breakdown, and is deployed on Render. The project demonstrates an end-to-end workflow from raw text to a deployed interactive application.

Keywords: Emotion classification, deep learning, NLP, BiGRU, GRU, text preprocessing, TensorFlow, FastAPI, web deployment.

TABLE OF CONTENTS

Chapter	Title
1	Introduction
2	Problem Statement and Objectives
3	Dataset and Exploratory Analysis
4	Methodology and Preprocessing
5	Model Architecture
6	Model Comparison and Results
7	System Design and Implementation
8	Deployment and Challenges
9	Limitations and Future Scope
10	Conclusion
—	References

Project links

Live demo: https://emotion-predictor-681h.onrender.com
Source repository: github.com/Beingwithpk/emotion-predictor

1. INTRODUCTION

Human communication conveys more than literal meaning. Word choice, phrasing, and context can indicate happiness, sadness, affection, fear, anger, or surprise. Emotion classification uses computational methods to assign text to one or more emotion categories. It is a useful application of natural language processing (NLP), with potential uses in feedback analysis, conversational interfaces, and text analytics.

Traditional keyword-based systems often struggle with context. For example, the word “happy” may appear in a negated sentence such as “I am not happy about this.” Deep-learning sequence models can learn patterns from word order and context, making them suitable for text classification tasks.

This project builds an end-to-end application rather than stopping at model training. A user enters a sentence into a web page; the frontend sends it to a backend API; the text is converted into the sequence format expected by the trained model; and the backend returns the predicted emotion and class probabilities. The interface then displays the prediction in a visual form.

1.1 Aim

To develop a web-based system that classifies a short English sentence into one of six emotion categories using a deep-learning model.

1.2 Scope

- Text-based classification for sadness, joy, love, anger, fear, and surprise.
- A recurrent neural network model based on bidirectional GRU layers.
- A REST API for inference and a browser interface for user interaction.
- Deployment on Render and documentation of the setup and operational challenges.

1.3 Report organization

The following chapters explain the problem, dataset, preprocessing pipeline, model design, comparison results, application architecture, deployment process, limitations, and possible extensions.

2. PROBLEM STATEMENT AND OBJECTIVES

Problem statement

Given a short English sentence, predict the most likely emotion from a fixed set of six labels. The model should return a readable emotion name and a confidence estimate, while the application should make inference accessible through a simple website.

Objectives

- Prepare and inspect a labelled emotion dataset.
- Normalize text and convert it into padded token sequences suitable for neural-network input.
- Train and compare Simple RNN, LSTM, GRU, and Bidirectional GRU architectures.
- Select a model based on the documented experimental results.
- Serve predictions through a FastAPI endpoint.
- Build a custom HTML/CSS/JavaScript interface that displays the predicted class and probabilities.
- Deploy the application and document setup issues encountered in a cloud environment.

Functional requirements

The application should accept text input, return a predicted emotion, provide a confidence score and probability distribution, show service/model status, and communicate errors when the model is loading or the server cannot be reached. The README also documents example sentences, a short session history, keyboard submission with Ctrl + Enter, and interactive API documentation at /docs.

Non-functional considerations

The system should be understandable to users, reasonably responsive after startup, and reproducible from the repository. Model-loading memory, cold starts on the free hosting tier, library compatibility, and case-sensitive paths are important operational considerations.

3. DATASET AND EXPLORATORY ANALYSIS

3.1 Dataset source

The project uses the DAIR.AI Emotion dataset hosted on Hugging Face. It contains short English text examples labelled with one of six emotions.

3.2 Dataset split and labels

Split / field	Documented details
Training split	Approximately 16,000 rows
Test split	Approximately 2,000 rows
Label set	Sadness, joy, love, anger, fear, surprise

3.3 Exploratory data analysis

The project documentation reports that a class-count plot revealed class imbalance, with joy as the largest class. Imbalanced labels can bias a classifier toward common categories and make aggregate accuracy alone an incomplete measure of model quality.

To address the imbalance during training, class weights were computed and applied. This increases the training penalty associated with mistakes on underrepresented classes, helping the model pay attention to classes that occur less often.

3.4 Label mapping

Integer label IDs were mapped back to their emotion names so that predictions can be interpreted by people. This mapping must remain consistent between training, evaluation, and the web API; otherwise, a correct class index could be displayed as the wrong emotion.

Data and evaluation note

The supplied README gives approximate split sizes and a qualitative description of the class distribution. It does not provide the exact class counts, dataset version/hash, or a class-wise metric table. These details should be added to a reproducibility appendix if they are available in the training notebooks or experiment logs.

4. METHODOLOGY AND PREPROCESSING

The workflow converts raw text into numerical sequences, learns representations of token context, and predicts a probability distribution over the six labels.

4.1 Text normalization

- Convert text to lowercase.
- Remove apostrophes and special characters according to the documented cleaning rules.
- Collapse repeated whitespace so the input is represented consistently.

Normalization reduces superficial differences between sentences. Preprocessing should be applied consistently during training and inference, because a mismatch can reduce prediction quality.

4.2 Tokenization and sequence construction

A Keras Tokenizer maps words to integer IDs. The vocabulary is limited to 10,000 words for the documented model settings. Each text is converted to a sequence and padded or truncated to a fixed length of 50 tokens. The README specifies end padding and truncation.

4.3 Class-weighted training

Because the class distribution is uneven, class weights are passed during training. This is intended to reduce the tendency to optimize mainly for the majority class. Class weighting does not guarantee balanced performance, so class-wise precision, recall, and F1 should still be inspected.

4.4 Training and evaluation workflow

- Prepare labelled train and test data.
- Apply the same text-cleaning and tokenizer pipeline.
- Train candidate recurrent architectures with shared settings where documented.
- Compare the recorded loss and accuracy values.
- Select the BiGRU model, then assess its overall result and class-level bias.
- Save the trained model and tokenizer for inference in the deployed API.

The README documents sparse categorical cross-entropy as the loss function. This is appropriate when each training example has one integer class label among the six mutually exclusive categories.

5. MODEL ARCHITECTURE

5.1 Selected model: Bidirectional GRU

The selected classifier is a stacked Bidirectional Gated Recurrent Unit (BiGRU) network. A GRU is a recurrent neural-network unit that uses update and reset gates to control how information is retained or replaced as a sequence is processed. A bidirectional layer processes the sequence in both forward and reverse directions, allowing the representation to use context from both sides of a token.

5.2 Layer configuration

Layer	Configuration	Purpose
Embedding	Vocabulary 10,000; 300 dimensions	Maps token IDs to dense vectors
Bidirectional GRU 1	128 units; returns sequences	Learns contextual features for each token
Dropout	0.5	Regularization to reduce overfitting
Bidirectional GRU 2	64 units	Summarizes contextual sequence information
Dropout	0.5	Additional regularization
Dense output	6 units; softmax	Produces probabilities for the six emotions

5.3 Output interpretation

The softmax layer converts the final representation into six class probabilities. The class with the largest probability is returned as the predicted emotion. The application also reports the confidence score and the complete probability breakdown, so users can see how strongly the model favours the selected category.

5.4 Why bidirectionality can help

Emotion depends on context and word order. In a sentence such as “I am not happy about this,” the negation changes the meaning of “happy.” A bidirectional model can build representations using words that occur before and after a token. GRU gates help preserve useful sequence information, while dropout is used to reduce overfitting. The project documentation also notes that GRUs have fewer parameters than LSTMs in many comparable configurations, which can make them lighter to train and serve.

Padding consideration

The README notes that the Simple RNN, LSTM, and unidirectional GRU baselines performed poorly, and identifies end padding as one possible setup issue: their final hidden state may follow a sequence of padding tokens. The bidirectional model can encounter the real words last in its reverse direction. This is a plausible explanation, not a proven cause; masking padding and rerunning the baselines is listed as future work.

6. MODEL COMPARISON AND RESULTS

6.1 Recorded comparison

The following values are reproduced from the supplied project README. The first three models performed poorly in the documented experiment, while the BiGRU was selected for the application.

Model	Loss	Accuracy	Documented verdict
Simple RNN	1.80	0.10	Poor
LSTM	1.82	0.112	Poor
GRU	1.77	0.118	Poor
Bidirectional GRU	**0.21**	**0.92**	**Chosen**

6.2 Interpretation

The reported BiGRU accuracy is 0.92 (92%) with loss 0.21. These figures indicate strong aggregate performance in the documented run, but they should be interpreted alongside the evaluation split and per-class results. The supplied README does not include precision, recall, F1 scores, a confusion matrix, or confidence intervals, so those values are not claimed in this report.

The unusually low accuracy of the three baselines suggests that the comparison may reflect differences in implementation or preprocessing rather than a fair comparison of the architectures themselves. The project README explicitly cautions that the baseline setup may have a padding-related issue. A controlled rerun with padding masks, consistent training conditions, and per-class metrics would make the comparison more reliable.

6.3 Example prediction

The API documentation includes the illustrative input “I feel so happy and excited,” with “joy” as the predicted emotion and a displayed confidence of 0.97. The README explicitly identifies these response values as illustrative; they should not be treated as a verified benchmark test case.

6.4 Recommended evaluation additions

- Report precision, recall, and F1 for every emotion class.
- Include a confusion matrix to show which emotions are commonly confused.
- State the exact test split and whether any hyperparameter selection used it.
- Repeat baseline training after masking padding and keep settings comparable.

7. SYSTEM DESIGN AND IMPLEMENTATION

7.1 High-level architecture

The application consists of a browser-based frontend, a FastAPI backend, and saved model artifacts. The frontend collects the sentence and sends a request to the backend. The backend preprocesses the input with the fitted tokenizer, creates the fixed-length sequence, invokes the trained BiGRU model, and returns a JSON response. The frontend uses that response to update the visual result.

1. User enters text
HTML/CSS/JavaScript interface
POST /predict request
FastAPI validates and preprocesses text
Tokenizer → padded sequence → BiGRU model
Predicted emotion + confidence + probabilities
Frontend displays emotion and probability bars

7.2 API endpoints

Method	Path	Purpose
GET	/	Serves the web interface
GET	/health	Reports server and model status
POST	/predict	Predicts the emotion of submitted text
GET	/docs	Interactive API documentation

7.3 Request and response format

A typical request sends a sentence as JSON. The backend returns the original text, predicted emotion, confidence, and a probability for each class. The documented backend response uses the field name `all_probabilites` (with that spelling), which the frontend also expects.

```
POST /predict\n{ "text": "I feel so happy and excited" }\n\nResponse fields:\n- text\n- predicted_emotion\n- co
```

7.4 User interface features

- Emotion-sensitive page colour, animated probability bars, and an emoji burst.
- One-click example sentences and a short session history.
- Ctrl + Enter keyboard shortcut for analysis.
- Status indicator and error messages for model loading or unreachable server.

8. DEPLOYMENT AND CHALLENGES

8.1 Technology stack

Component	Technology	Role
Deep-learning model	TensorFlow / Keras; BiGRU	Text classification inference
Tokenization	Keras Tokenizer	Maps words to integer sequences
Backend	FastAPI, Uvicorn, Pydantic	HTTP endpoints and request handling
Frontend	HTML, CSS, JavaScript	Input, visual feedback, and interaction
Hosting	Render	Public web service hosting

8.2 Repository structure

```
emotion-predictor/\n■■■ main.py                # FastAPI app and model loading\n■■■ Artifacts/\n■   ■■■ BIG
```

8.3 Local execution

The README documents cloning the GitHub repository, creating and activating a Python virtual environment, installing dependencies from requirements.txt, and starting the application with Uvicorn. The local interface is then available at http://127.0.0.1:8000.

```
git clone https://github.com/Beingwithpk/emotion-predictor\ncd emotion-predictor\npython -m venv .venv\n# Windo
```

8.4 Cloud deployment

The application is deployed as a Render web service connected to the GitHub repository. The documented build command is “pip install -r requirements.txt”; the start command runs Uvicorn on 0.0.0.0 using the port supplied by the hosting environment. The README recommends setting PYTHON_VERSION to 3.11.9 because the documented TensorFlow setup did not support the platform’s default Python 3.14.

8.5 Deployment lessons

- Python compatibility: use a supported Python version for the installed TensorFlow release.
- Case-sensitive paths: Linux distinguishes names such as Static and static.
- Model/library compatibility: use stable TensorFlow/Keras versions compatible with the saved model.
- Consistent response keys: backend and frontend must use the same JSON field spelling.
- Repository completeness: ensure model artifacts and static assets are committed.
- Cold starts and memory: a free instance may sleep while idle and can require time to load the model.

9. LIMITATIONS AND FUTURE SCOPE

9.1 Current limitations

- The model predicts one dominant class even when a sentence expresses multiple emotions.
- The dataset contains short English sentences, so performance may not transfer to other languages, long documents, slang-heavy text, or domain-specific writing.
- Class imbalance can affect individual emotion categories even when overall accuracy is high.
- The report has no documented per-class precision, recall, F1 scores, or confusion matrix.
- The baseline comparison may be affected by padding or setup issues and should not be interpreted as a definitive ranking of architectures.
- Render free-tier sleep behaviour can cause delayed first requests, and model loading may be constrained by available memory.

9.2 Future improvements

- Evaluate a held-out test set with per-class precision, recall, F1, and a confusion matrix.
- Mask padding and retrain the Simple RNN, LSTM, and GRU baselines under comparable conditions.
- Compare the BiGRU against a pretrained transformer such as DistilBERT.
- Explore a lighter model format or optimized inference path to reduce memory use and startup time.
- Extend the formulation to multi-label emotion classification so that a sentence can express more than one emotion.
- Record exact dataset version, preprocessing configuration, random seeds, training settings, and package versions for reproducibility.

9.3 Responsible interpretation

Predictions should be treated as model estimates, not definitive statements about a writer’s internal emotional state. The same sentence can be interpreted differently depending on context, culture, sarcasm, or personal circumstances. The tool is best used as an assistive text-analysis feature, with human judgment retained for consequential decisions.

10. CONCLUSION

This project presents an end-to-end emotion classification application that maps short English text to one of six emotion categories using a Bidirectional GRU deep-learning model. The workflow includes dataset exploration, label mapping, text normalization, tokenization, sequence padding, class-weighted training, architecture comparison, API development, frontend design, and cloud deployment.

According to the supplied project documentation, the selected BiGRU achieved a reported accuracy of 0.92 and loss of 0.21, outperforming the recorded Simple RNN, LSTM, and GRU runs. The README also acknowledges that the baseline comparison may be affected by padding behaviour, so additional controlled experiments and class-wise metrics are necessary before making stronger comparative claims.

The FastAPI service makes the model accessible through a /predict endpoint, while the browser interface presents the prediction, confidence, and probability breakdown. Deployment work highlighted the practical importance of supported Python versions, compatible model libraries, case-sensitive file paths, correct response keys, committed model artifacts, and memory-aware hosting.

Overall, the project demonstrates how a deep-learning model can be integrated into a usable web application. Future work should focus on stronger evaluation, a fairer baseline comparison, lighter inference, and support for sentences that express multiple emotions.

REFERENCES

1. DAIR.AI, “Emotion” dataset, Hugging Face Datasets. https://huggingface.co/datasets/dair-ai/emotion

2. TensorFlow, “TensorFlow documentation.” https://www.tensorflow.org/

3. Keras, “Keras documentation.” https://keras.io/

4. FastAPI, “FastAPI documentation.” https://fastapi.tiangolo.com/

5. Uvicorn, “Uvicorn documentation.” https://www.uvicorn.org/

6. Render, “Web services documentation.” https://render.com/docs/web-services

7. Project repository: Beingwithpk, “emotion-predictor.” https://github.com/Beingwithpk/emotion-predictor

8. Project live demo: https://emotion-predictor-681h.onrender.com

End of Report